

MongoDB Tutorial

A hands-on reference for the `mongosh` shell, rewritten so every command works on current MongoDB. The legacy `mongo` commands are mapped to their replacements.

Shell	<code>mongosh</code> 2.12.0 — the <code>mongo</code> shell was removed in MongoDB 6.0
Server	MongoDB 9.0.2 (Community), running locally on 127.0.0.1:27017
Examples	All 48 commands in this document were executed against that server; every <i>output</i> block is the real response, not a screenshot
Updated	04 October 2026

What changed since the previous version

The previous tutorial was written for the `mongo` shell. That shell was deprecated in MongoDB 5.0 and removed in 6.0, and several collection helpers were deprecated along with it. Each row below was tested on the versions named on the cover.

OLD COMMAND	USE INSTEAD	STATUS TODAY	WHY
mongo	mongosh	✗ Removed	Legacy shell, removed in MongoDB 6.0. Shell is now <code>mongosh</code> .
mongo -u root -p root -host localhost	mongosh	✗ Fails	A local server installed for learning has no authentication, so credentials are rejected with <code>Authentication failed</code> .
db.c.insert(doc)	db.c.insertOne(doc)	⚠ Deprecated	Still runs in mongosh 2.12, but deprecated.
db.c.insertmany([...])	db.c.insertMany([...])	✗ Fails	Lowercase <code>m</code> throws <code>is not a function</code> . The method is <code>insertMany</code> .
db.c.save(doc)	db.c.insertOne / replaceOne	✗ Removed	Removed from mongosh: <code>is not a function</code> .
db.c.find().count()	db.c.countDocuments(filter)	⚠ Deprecated	Runs, but prints a driver deprecation warning.
db.c.update(f, d)	db.c.updateOne / updateMany / replaceOne	⚠ Deprecated	Runs, but prints <code>DeprecationWarning: Collection.update() is deprecated</code> .
db.c.remove(f)	db.c.deleteOne / deleteMany	⚠ Deprecated	Runs, but prints <code>DeprecationWarning: Collection.remove() is deprecated</code> .
db.c.ensureIndex({f: 1})	db.c.createIndex({f: 1})	⚠ Deprecated	Deprecated alias; still works.
db.c.find().pretty()	db.c.find()	○ Not needed	mongosh formats output already. <code>.pretty()</code> is harmless but redundant.

NOTE **Deprecated** means it still runs today but prints a warning and will be removed. **Fails** and **removed** mean it throws an error now. Teaching the new names costs nothing and they are clearer: the method says how many documents it touches.

Contents

1. Introduction
2. The shell: mongosh
3. Databases and collections
4. Create documents
5. Read documents
6. Update documents
7. Delete documents
8. Sub-documents and arrays
9. Indexes
10. Comparison and logical operators
11. Shell functions
12. Aggregation, in one example
13. Cleaning up

Introduction

MongoDB is a NoSQL database. It stores data in collections that contain documents (also called records). Each document is a JSON-like object: a set of fields in key-value pairs, and a document may contain sub-documents hierarchically. Internally MongoDB stores these documents in a binary-encoded format called BSON, which adds types that plain JSON does not have, such as ObjectId, Date and Decimal128.

Compared with a relational database: a **collection** is roughly a table, a **document** is roughly a row, and a **field** is roughly a column. Unlike a table, documents in one collection are not required to share the same shape.

The shell: mongosh

Every mongo command from the old tutorial still has a direct replacement. The table on page 2 lists them all.

The old `mongo` shell was deprecated in MongoDB 5.0 and **removed in MongoDB 6.0**. The shell is now `mongosh` (MongoDB Shell). Every command in this tutorial is written for `mongosh`.

```
mongosh
```

Connects to `mongodb://127.0.0.1:27017` — the default. This is all you need for a local server.

```
mongosh --host localhost --port 27017
mongosh "mongodb://localhost:27017"
mongosh "mongodb://localhost:27017/blog"
```

Equivalent forms. The last one also selects the `blog` database on connect.

NOTE A local server installed for learning has **no authentication**, so do not pass `-u / -p`. Passing credentials to a server without auth fails with `MongoServerError: Authentication failed`. Only use `-u root -p root` when the server was actually started with authentication enabled and that user exists, which is the case for the usual Docker Compose setup (`MONGO_INITDB_ROOT_USERNAME`).

```
mongosh -u root -p root --authenticationDatabase admin --host localhost
```

Only for a server that has authentication enabled, such as the Docker image configured with a root user.

```
docker run -d -p 27017:27017 --name mongo mongo:8
docker exec -it mongo mongosh
```

Alternative to a local install: run the server in Docker. The image bundles `mongosh`.

Databases and collections

```
use blog
```

Selects (and lazily creates) a database. After this, commands start with `db.`. `use` is a shell command, so it has no parentheses.

```
db
```

OUTPUT

```
blog
```

Shows the current database.

```
show dbs
```

OUTPUT

```
admin    40.00 KiB
config  116.00 KiB
local    40.00 KiB
```

Lists databases. A database with no collections is not listed.

```
db.createCollection('posts')
```

OUTPUT

```
{ ok: 1 }
```

Creates a collection explicitly.

```
show collections
```

OUTPUT

```
posts
```

Lists collections in the current database.

NOTE You rarely need `createCollection`: inserting into a collection that does not exist creates it. Create it explicitly when you want options such as a schema validator or a capped collection.

Create documents

```
db.posts.insertOne({ title: 'Post One', author: 'Dana', likes: 4, tags: ['mongo', 'intro'] })
```

OUTPUT

```
{
  acknowledged: true,
  insertedId: ObjectId('6ac24d6425c8d787993d87f7')
}
```

Inserts one document and returns its generated `_id`.

```
db.posts.insertMany([
  { title: 'Post Two', author: 'Omer', likes: 12, tags: ['mongo'] },
  { title: 'Post Three', author: 'Dana', likes: 7, tags: ['node', 'mongo'] },
  { title: 'Post Four', author: 'Noa', likes: 0, tags: ['intro'] },
  { title: 'Post Five', author: 'Omer', likes: 25, tags: ['node'] }
])
```

OUTPUT

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac24d650848ff1d6e262c32'),
    '1': ObjectId('6ac24d650848ff1d6e262c33'),
    '2': ObjectId('6ac24d650848ff1d6e262c34'),
    '3': ObjectId('6ac24d650848ff1d6e262c35')
  }
}
```

Inserts several documents in one call. Note the capital M in `insertMany`.

LEGACY

~~`db.collection.insert(...)`~~ → `db.collection.insertOne(...)` / `insertMany(...)`

`insert()` still runs in mongosh 2.12 but is deprecated; `insertmany()` (lowercase m) is **not a function** and throws, and `save()` was removed entirely.

Read documents

```
db.posts.find()
```

OUTPUT

```
[
  {
    _id: ObjectId('6ac24d6425c8d787993d87f7'),
    title: 'Post One',
    author: 'Dana',
    likes: 4,
    tags: [ 'mongo', 'intro' ]
  },
  {
    _id: ObjectId('6ac24d650848ff1d6e262c32'),
    title: 'Post Two',
    author: 'Omer',
    likes: 12,
    tags: [ 'mongo' ]
  },
  {
    _id: ObjectId('6ac24d650848ff1d6e262c33'),
    title: 'Post Three',
    author: 'Dana',
    likes: 7,
    tags: [ 'node', 'mongo' ]
  },
  {
    _id: ObjectId('6ac24d650848ff1d6e262c34'),
    title: 'Post Four',
  },
]
```

```

    author: 'Noa',
    likes: 0,
    tags: [ 'intro' ]
  },
  {
    _id: ObjectId('6ac24d650848ff1d6e262c35'),
    title: 'Post Five',
    author: 'Omer',
    likes: 25,
    tags: [ 'node' ]
  }
]

```

All documents in the collection.

NOTE mongosh already prints formatted, colorized output, so `.pretty()` is no longer needed. It still exists and does no harm, but you can drop it.

```
db.posts.find({ author: 'Dana' })
```

OUTPUT

```

[
  {
    _id: ObjectId('6ac24d6425c8d787993d87f7'),
    title: 'Post One',
    author: 'Dana',
    likes: 4,
    tags: [ 'mongo', 'intro' ]
  },
  {
    _id: ObjectId('6ac24d650848ff1d6e262c33'),
    title: 'Post Three',
    author: 'Dana',
    likes: 7,
    tags: [ 'node', 'mongo' ]
  }
]

```

Documents matching a condition.

```
db.posts.find({}, { title: 1, likes: 1, _id: 0 })
```

OUTPUT

```

[
  { title: 'Post One', likes: 4 },
  { title: 'Post Two', likes: 12 },
  { title: 'Post Three', likes: 7 },
  { title: 'Post Four', likes: 0 },
  { title: 'Post Five', likes: 25 }
]

```

A projection: return only these fields. `_id: 0` suppresses the id.

```
db.posts.find().sort({ likes: -1 })
```

OUTPUT

```

[
  {
    _id: ObjectId('6ac24d650848ff1d6e262c35'),

```

```

    title: 'Post Five',
    author: 'Omer',
    likes: 25,
    tags: [ 'node' ]
  },
  {
    _id: ObjectId('6ac24d650848ff1d6e262c32'),
    title: 'Post Two',
    author: 'Omer',
    likes: 12,
    tags: [ 'mongo' ]
  },
  {
    _id: ObjectId('6ac24d650848ff1d6e262c33'),
    title: 'Post Three',
    author: 'Dana',
    likes: 7,
    tags: [ 'node', 'mongo' ]
  },
  {
    _id: ObjectId('6ac24d6425c8d787993d87f7'),
    title: 'Post One',
    author: 'Dana',
    likes: 4,
    tags: [ 'mongo', 'intro' ]
  },
  {
    _id: ObjectId('6ac24d650848ff1d6e262c34'),
    title: 'Post Four',
    author: 'Noa',
    likes: 0,
    tags: [ 'intro' ]
  }
]

```

Sort descending. Use `1` for ascending.

```
db.posts.findOne({ author: 'Omer' })
```

OUTPUT

```

{
  _id: ObjectId('6ac24d650848ff1d6e262c32'),
  title: 'Post Two',
  author: 'Omer',
  likes: 12,
  tags: [ 'mongo' ]
}

```

The first matching document, as an object rather than a cursor.

```
db.posts.countDocuments()
```

OUTPUT

```
5
```

Counts documents.

```
db.posts.countDocuments({ author: 'Omer' })
```

OUTPUT

2

Counts documents matching a condition.

LEGACY

~~db.collection.find().count()~~ → `db.collection.countDocuments(filter)`

`count()` still returns a number but prints a driver deprecation warning. `estimatedDocumentCount()` is the fast, approximate variant that uses collection metadata.

```
db.posts.find().limit(2)
```

OUTPUT

```
[
  {
    _id: ObjectId('6ac24d6425c8d787993d87f7'),
    title: 'Post One',
    author: 'Dana',
    likes: 4,
    tags: [ 'mongo', 'intro' ]
  },
  {
    _id: ObjectId('6ac24d650848ff1d6e262c32'),
    title: 'Post Two',
    author: 'Omer',
    likes: 12,
    tags: [ 'mongo' ]
  }
]
```

Limits the number of results.

```
db.posts.find().sort({ likes: -1 }).skip(1).limit(2)
```

OUTPUT

```
[
  {
    _id: ObjectId('6ac24d650848ff1d6e262c32'),
    title: 'Post Two',
    author: 'Omer',
    likes: 12,
    tags: [ 'mongo' ]
  },
  {
    _id: ObjectId('6ac24d650848ff1d6e262c33'),
    title: 'Post Three',
    author: 'Dana',
    likes: 7,
    tags: [ 'node', 'mongo' ]
  }
]
```

Skip plus limit is how you paginate.

Update documents

```
db.posts.updateOne({ title: 'Post Four' }, { $set: { likes: 3 } })
```

OUTPUT

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Updates the first matching document. `$set` changes only the listed fields and leaves the rest intact.

NOTE Without an operator such as `$set`, the document would be **replaced** entirely and the unlisted fields would be lost. That is what `replaceOne` is for, and it is the most common beginner mistake with the old `update()` helper.

```
db.posts.updateOne({ title: 'Post One' }, { $inc: { likes: 5 } })
```

OUTPUT

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

`$inc` increments a numeric field. A negative value decrements it.

```
db.posts.updateOne({ title: 'Post Six' }, { $set: { author: 'Guest', likes: 1 } }, { upsert: true })
```

OUTPUT

```
{
  acknowledged: true,
  insertedId: ObjectId('6ac24d6a07ddd7cec9a41833'),
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
```

`upsert: true` inserts the document when nothing matches the filter.

```
db.posts.updateMany({ author: 'Omer' }, { $set: { verified: true } })
```

OUTPUT

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
```

Updates every matching document. Pass `{}` as the filter to update all.

```
db.posts.updateMany({}, { $rename: { 'tags': 'labels' } })
```

OUTPUT

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 6,
  modifiedCount: 5,
  upsertedCount: 0
}
```

`$rename` renames a field.

```
db.posts.updateMany({}, { $rename: { 'labels': 'tags' } })
```

OUTPUT

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 6,
  modifiedCount: 5,
  upsertedCount: 0
}
```

Renamed back, so the rest of the tutorial keeps using `tags`.

```
db.posts.replaceOne({ title: 'Post Six' }, { title: 'Post Six', author: 'Guest', likes: 2 })
```

OUTPUT

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Replaces the whole document, keeping the same `_id`.

LEGACY

~~db.collection.update(filter, doc, { upsert: true })~~ →

db.collection.updateOne / updateMany / replaceOne

update() still runs but prints DeprecationWarning: Collection.update() is deprecated . It also silently updated only the first match, which surprised people; the new names say what they do.

Delete documents

```
db.posts.deleteOne({ title: 'Post Six' })
```

OUTPUT

```
{ acknowledged: true, deletedCount: 1 }
```

Deletes the first matching document.

```
db.posts.insertMany([{ title: 'Draft A', status: 'draft' }, { title: 'Draft B', status: 'draft' }])
```

OUTPUT

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac24d6d0c643386b3946c95'),
    '1': ObjectId('6ac24d6d0c643386b3946c96')
  }
}
```

Two throwaway documents, so the next command has something to delete.

```
db.posts.deleteMany({ status: 'draft' })
```

OUTPUT

```
{ acknowledged: true, deletedCount: 2 }
```

Deletes every matching document, and reports how many.

NOTE deleteMany({}) with an empty filter deletes **every document** in the collection and does not ask for confirmation. There is no undo.

LEGACY

~~db.collection.remove(filter)~~ → db.collection.deleteOne / deleteMany

remove() still runs but prints DeprecationWarning: Collection.remove() is deprecated .

db.collection.drop() removes the whole collection, and db.dropDatabase() the database.

Sub-documents and arrays

A field can hold an object or an array of objects. This is what makes MongoDB different from a relational table: data that would need a join often lives inside the document.

```
db.posts.updateOne({ title: 'Post One' }, { $set: { comments: [
  { user: 'Noa', text: 'Great post', stars: 5 },
  { user: 'Omer', text: 'Thanks', stars: 3 }
] } })
```

OUTPUT

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Adds an array of sub-documents.

```
db.posts.find({ 'comments.user': 'Noa' }, { title: 1, _id: 0 })
```

OUTPUT

```
[ { title: 'Post One' } ]
```

Dot notation matches a field inside a sub-document. Quote the key because it contains a dot.

```
db.posts.find({ comments: { $elemMatch: { user: 'Omer', stars: { $lt: 4 } } } }, { title: 1, _id: 0 })
```

OUTPUT

```
[ { title: 'Post One' } ]
```

`$elemMatch` requires **one single** array element to match all the conditions. Without it, different elements could satisfy different conditions.

```
db.posts.updateOne({ title: 'Post One' }, { $push: { comments: { user: 'Dana', text: 'Fixed', stars: 4 } } })
```

OUTPUT

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

`$push` appends to an array; `$pull` removes matching elements and `$addToSet` appends only if absent.

Indexes

An index makes a query fast and is required for text search. Without a text index, searching for a word inside a string field does not work at all.

```
db.posts.getIndexes()
```

OUTPUT

```
[
  {
    v: 2,
    key: { _id: 1 },
    name: '_id_',
    collation: { locale: 'simple' }
  }
]
```

Before creating anything, only the automatic `_id` index exists.

```
db.posts.createIndex({ title: 'text' })
```

OUTPUT

```
title_text
```

Creates a text index on `title`.

```
db.posts.find({ $text: { $search: 'Five' } }, { title: 1, _id: 0 })
```

OUTPUT

```
[ { title: 'Post Five' } ]
```

Now a word search works.

```
db.posts.find({ $text: { $search: '\"Post Three\"' } }, { title: 1, _id: 0 })
```

OUTPUT

```
[ { title: 'Post Three' } ]
```

Escaped double quotes inside the search string match an exact phrase.

```
db.posts.createIndex({ author: 1 })
```

OUTPUT

```
author_1
```

A regular ascending index, for filtering and sorting by author.

```
db.posts.getIndexes()
```

OUTPUT

```
[
  {
    v: 2,
    key: { _id: 1 },
    name: '_id_',
    collation: { locale: 'simple' }
  },
  {
    v: 1,
    key: { author: 1 },
    name: 'author_1',
    collation: { locale: 'simple' }
  }
]
```

```

},
{
  v: 2,
  key: { _fts: 'text', _ftsx: 1 },
  name: 'title_text',
  weights: { title: 1 },
  default_language: 'english',
  language_override: 'language',
  textIndexVersion: 3,
  collation: { locale: 'simple' }
},
{
  v: 2,
  key: { author: 1 },
  name: 'author_1',
  collation: { locale: 'simple' }
}
]

```

All indexes on the collection.

LEGACY

~~db.collection.ensureIndex({ f: 1 })~~ → db.collection.createIndex({ f: 1 })

ensureIndex() is a deprecated alias that still works. createIndex() is the current name.

Comparison and logical operators

```
db.posts.find({ likes: { $gt: 7 } }, { title: 1, likes: 1, _id: 0 })
```

OUTPUT

```

[
  { title: 'Post One', likes: 9 },
  { title: 'Post Two', likes: 12 },
  { title: 'Post Five', likes: 25 }
]

```

\$gt : greater than.

```
db.posts.find({ likes: { $gte: 9 } }, { title: 1, likes: 1, _id: 0 })
```

OUTPUT

```

[
  { title: 'Post One', likes: 9 },
  { title: 'Post Two', likes: 12 },
  { title: 'Post Five', likes: 25 }
]

```

\$gte : greater than or equal.

```
db.posts.find({ likes: { $lt: 10 } }, { title: 1, likes: 1, _id: 0 })
```

OUTPUT

```
[
  { title: 'Post One', likes: 9 },
  { title: 'Post Three', likes: 7 },
  { title: 'Post Four', likes: 3 }
]
```

`$lt`: less than.

```
db.posts.find({ likes: { $lte: 7 } }, { title: 1, likes: 1, _id: 0 })
```

OUTPUT

```
[ { title: 'Post Three', likes: 7 }, { title: 'Post Four', likes: 3 } ]
```

`$lte`: less than or equal.

```
db.posts.find({ likes: { $gte: 5, $lte: 20 } }, { title: 1, likes: 1, _id: 0 })
```

OUTPUT

```
[
  { title: 'Post One', likes: 9 },
  { title: 'Post Two', likes: 12 },
  { title: 'Post Three', likes: 7 }
]
```

A range: both bounds in one object.

```
db.posts.find({ author: { $in: ['Dana', 'Noa'] } }, { title: 1, author: 1, _id: 0 })
```

OUTPUT

```
[
  { title: 'Post One', author: 'Dana' },
  { title: 'Post Three', author: 'Dana' },
  { title: 'Post Four', author: 'Noa' }
]
```

`$in` matches any value in a list; `$nin` is its negation.

```
db.posts.find({ author: { $ne: 'Omer' } }, { title: 1, author: 1, _id: 0 })
```

OUTPUT

```
[
  { title: 'Post One', author: 'Dana' },
  { title: 'Post Three', author: 'Dana' },
  { title: 'Post Four', author: 'Noa' }
]
```

`$ne`: not equal.

```
db.posts.find({ $or: [{ author: 'Noa' }, { likes: { $gt: 20 } }] }, { title: 1, author: 1, likes: 1, _id: 0 })
```

OUTPUT

```
[
  { title: 'Post Four', author: 'Noa', likes: 3 },
  { title: 'Post Five', author: 'Omer', likes: 25 }
]
```

`$or` takes an array of conditions. `$and` is implicit when you list several fields.

```
db.posts.find({ verified: { $exists: false } }, { title: 1, _id: 0 })
```

OUTPUT

```
[
  { title: 'Post One' },
  { title: 'Post Three' },
  { title: 'Post Four' }
]
```

`$exists` tests for the presence of a field, which matters when documents have different shapes.

Shell functions

```
print('hello from mongosh')
```

OUTPUT

```
hello from mongosh
```

`print()` writes to the shell.

```
db.posts.find().forEach(p => print(p.title + ' by ' + p.author))
```

OUTPUT

```
Post One by Dana
Post Two by Omer
Post Three by Dana
Post Four by Noa
Post Five by Omer
```

`forEach` runs a function per document. mongosh is JavaScript, so arrow functions, `const` and template literals all work.

```
db.posts.find({ author: 'Dana' }).toArray().map(p => p.title)
```

OUTPUT

```
[ 'Post One', 'Post Three' ]
```

`toArray()` turns a cursor into a real array, so array methods such as `map` and `filter` become available.

Aggregation, in one example

Aggregation runs documents through a pipeline of stages. It is how you group and compute, the equivalent of SQL's GROUP BY. This was missing from the original tutorial and is worth knowing early.

```
db.posts.aggregate([
  { $match: { likes: { $gte: 0 } } },
  { $group: { _id: '$author', posts: { $sum: 1 }, totalLikes: { $sum: '$likes' } } },
  { $sort: { totalLikes: -1 } }
])
```

OUTPUT

```
[
  { _id: 'Omer', posts: 2, totalLikes: 37 },
  { _id: 'Dana', posts: 2, totalLikes: 16 },
  { _id: 'Noa', posts: 1, totalLikes: 3 }
]
```

Counts posts and sums likes per author, highest first.

Cleaning up

```
db.posts.drop()
db.dropDatabase()
```

`drop()` removes one collection; `dropDatabase()` removes the current database. Both are irreversible, so check `db` first to confirm which database you are on.